

Foundations of Visual Perception for Robotics: A Review of Color Spaces, Gaussian Pyramids, Optical Flow, GrabCut, and Supervised Learning

Aidan J. Martin

ENGR 6010: AI in Robotics, Middle Tennessee State University

Abstract. A camera gives a robot a grid of color values. Turning that grid into decisions requires answers to five questions: how to represent color, how to handle image structure at different scales, how to measure motion between frames, how to separate an object from its background, and how to learn a mapping from images to labels. This paper reviews one method for each question, namely color spaces, Gaussian pyramids, optical flow, GrabCut, and supervised learning, using the papers read in this course. Each review pairs the theory with a demonstration program from this course, most of them run on synthetic data with known ground truth, so that each method's failure condition could be measured rather than described. The five methods share a pattern: each fails at a specific, computable condition, and several fail without signaling it.

1. Introduction

Robots that work near people increasingly rely on cameras in place of dedicated sensors. A hand-held laparoscopic instrument records no kinematics, so its motion must be recovered from video [4], and a quadruped navigating a room must find its target and judge its own motion from images. Both tasks pass through the same chain of operations. The image arrives as red, green, and blue intensities, and the choice of color space decides which differences between pixels are easy to measure. Fine texture and large motion appear at different scales, and a Gaussian pyramid lets one algorithm work at all of them. Optical flow estimates where each pixel moved between frames. Segmentation methods such as GrabCut decide which pixels belong to an object. Supervised learning replaces hand-designed rules with mappings fit to labeled examples, and modern detectors build it on top of the earlier steps; the surgical keypoint detector of Ghanekar et al. [3], for example, takes optical flow as an input to a segmentation network trained on labeled frames. Section 2 reviews each method in this order, and Section 3 summarizes what the methods share.

2. Review of Methods

2.1 Color Spaces

A color model maps the light a sensor records to a small set of numerical components, usually three or four [1]. Ibraheem et al. sort color models into three groups [1]. Device-oriented models, such as RGB, CMY(K), YUV, YIQ, and YCbCr, depend on the hardware that captures or displays the color. User-oriented models, the HSI, HSV, and HSL family, use cylindrical coordinates that separate intensity from hue and saturation, which match how people describe color. Device-independent models, CIE XYZ, CIELUV, and CIELAB, follow colorimetric standards, and CIELUV and CIELAB are built so that equal numerical distances correspond to roughly equal perceived differences. RGB needs no conversion for display, so most images begin there, but the review notes that RGB is poor for specifying and recognizing a particular color [1]. The HSV family addresses this by isolating hue, which can drive a segmentation by itself, but hue becomes unstable and undefined for achromatic (gray) pixels [1]. Both sides of this trade-off appeared in this course. The GrabCut demonstration models foreground and background as clusters of RGB values, so the color space directly determines how well the two clusters separate. The dense optical flow visualization in Homework 1 uses the HSV color wheel in the other direction, encoding motion direction as hue and motion magnitude as saturation and value, so that a flow field can be read at a glance.

2.2 Gaussian Pyramids

A Gaussian pyramid represents an image at a sequence of resolutions. Each level blurs the previous level with a Gaussian filter and keeps every second pixel in each direction, halving the width and height [2]. The blur removes detail that the coarser grid cannot represent, so each level is a faithful low-resolution

copy rather than an aliased one. Szeliski presents the pyramid as the standard coarse-to-fine strategy for motion estimation [2, Sec. 9.1.1]: a displacement of d pixels at full resolution becomes $d/2^L$ pixels at level L , so a method that only works for small motion can run at the top of the pyramid, and its estimate can be doubled and refined at each finer level. The demonstration from the first reading assignment tested this directly. A four-level pyramid reduced a 15 pixel motion to about 1.9 pixels at the coarsest level, and pyramidal Lucas–Kanade recovered a 12×9 pixel shift exactly, as (12.00, 9.00), in 0.07 s. Exhaustive block matching reached the same accuracy in 8.97 s, 128 times longer, and only because its search range was set large enough in advance. The pyramid has one cost: a sample point must stay inside the image at every level, so the usable sample grid shrank from 875 points to 264.

2.3 Optical Flow

Optical flow assigns each pixel a velocity (u, v) between two frames. It rests on two assumptions: a point keeps its brightness as it moves, and the motion between frames is small. A first-order Taylor expansion of the brightness constancy condition $I(x + \delta x, y + \delta y, t + \delta t) = I(x, y, t)$ gives the optical flow constraint $I_x u + I_y v + I_t = 0$, where I_x , I_y , and I_t are the image derivatives in x , y , and time [2]. This is one equation in two unknowns, so only the motion component normal to the local edge is observable, which is the aperture problem. Lucas–Kanade resolves it by assuming the flow is constant over a small window and solving the least-squares system $A^T A v = A^T b$, where $A^T A$ is the 2×2 structure tensor built from products of gradients over the window [2, Sec. 9.1.3]. The system is solvable only when both eigenvalues of the structure tensor are large, which is the Shi–Tomasi criterion for a trackable corner. Homework 1 gave a concrete case: the structure tensor $\begin{bmatrix} 100 & 80 \\ 80 & 64 \end{bmatrix}$ has a determinant of zero and eigenvalues of 164 and 0, so every gradient in the patch points the same way and its flow cannot be recovered. The first reading assignment compared Lucas–Kanade with the block matching search in the Southampton interactive demonstration [9], which tests every integer offset in a range and scores each with the sum of squared differences. Block matching cannot represent a half-pixel motion, so its error never fell below 0.5 px, while iterative Lucas–Kanade recovered a (2.5, 1.5) shift to a mean error of 0.0148 px with one-fifth the operations per point. The linearization limits Lucas–Kanade in turn: a single solve exceeded 1 px of error once the true motion passed 4 px, which is the failure the pyramid in Section 2.2 repairs. Learned estimators such as RAFT now produce dense flow fields, and Ghanekar et al. feed RAFT flow into their surgical keypoint detector as temporal context [3].

2.4 GrabCut

GrabCut, by Rother, Kolmogorov, and Blake [5], extracts a foreground object from a single bounding box drawn by the user. Pixels outside the box are fixed as background and pixels inside are unknown. Each pixel receives a binary label, and the method minimizes an energy $E = U + V$. The data term U scores each pixel's color under one of two Gaussian mixture models (GMMs), one for the foreground and one for the background, each with five full-covariance components in RGB space. The smoothness term V adds a penalty for each pair of neighboring pixels that receive different labels, and the penalty decays exponentially with the color difference between them, so a cut between similar colors is expensive and a cut along a strong edge is cheap. Because the labels are binary and the pairwise penalty applies only when neighbors disagree, the minimum of E equals the minimum cut of a graph with one node per pixel and two terminal nodes, which a max-flow algorithm finds exactly. The max-flow min-cut theorem, which the network flow visualizer demonstrates one augmenting path at a time [10], is what makes this problem tractable. GrabCut then alternates between refitting the GMMs to the current labels and recomputing the cut, and the energy never increases. Peng, Zhang, and Zhang place GrabCut in the family of interactive graph cuts on Markov random field (MRF) models, in which each pixel's label depends on its neighbors [6]. Their benchmark on the Berkeley segmentation dataset found that iterated graph cut, GrabCut's category, gave the best accuracy among the interactive methods tested while needing the fewest user interactions, and they note that a fixed pairwise smoothness term cannot express a prior on object shape [6]. The demonstration from the second reading assignment animated this energy on a toy graph, then

stepped OpenCV's implementation one iteration at a time on a real image, which exposes the four-state mask (definite and probable foreground and background) described in the OpenCV documentation [11].

2.5 Supervised Learning

Kotsiantis defines supervised learning as the search for algorithms that reason from labeled instances to general hypotheses, which then predict future instances [7]. The review frames it as a process: collect data, preprocess it and select features, choose an algorithm, train it, and evaluate it on data held out from training, returning to earlier steps when the error is too high. It organizes classification algorithms into logic-based methods such as decision trees, perceptron-based methods including neural networks, statistical methods such as naive Bayes and Bayesian networks, instance-based methods such as k-nearest neighbors, and support vector machines, and it concludes that no single algorithm performs best on every problem [7]. Bottou, Curtis, and Nocedal state the training step as an optimization problem: choose the parameters that minimize the average loss over the training samples, called the empirical risk, as a stand-in for the expected loss over all possible data [8]. They develop the stochastic gradient method for this problem and survey second-order alternatives, including quasi-Newton methods that build a curvature estimate from pairs of parameter and gradient changes [8]. The demonstration from the third reading assignment showed why curvature matters: gradient descent zigzags along an ill-conditioned cost surface while Newton's method reaches the minimum directly, and BFGS learns the curvature from observed gradient changes. The gap between empirical and expected risk is where supervised perception fails in practice. The detector of Ghanekar et al. trains a DeepLabv3 segmentation network on labeled surgical frames and reports over 91% keypoint detection accuracy on its own benchmark [3], but our SPIE study found that holding out one surgical task dropped its percentage of correct keypoints (PCK) from 84.3% to 70.9%, and moving to a different dataset dropped it to 3.9% [4].

3. Conclusion

The five methods form one pipeline. A color space defines what the pixel values mean, a pyramid lets the same algorithm work at every scale, optical flow measures motion, GrabCut turns color models into a segmentation, and supervised learning fits the mapping when no hand-designed rule is good enough. Each method also has a specific condition under which it fails: hue at gray pixels, Lucas–Kanade on a rank-one structure tensor or on motion beyond a few pixels, block matching on motion outside its search range, GrabCut on objects that need a shape prior, and a trained network on data unlike its training set. Several of these failures return an ordinary-looking answer instead of an error. Saturated block matching, for example, produced a flow field that looked correct while missing the true motion. Most of the demonstrations in this course measured these conditions on synthetic data, where the ground truth is known exactly, and the same approach will guide data collection and simulation for the Unitree Go2 work that follows.

References

- [1] N. A. Ibraheem, M. M. Hasan, R. Z. Khan, and P. K. Mishra, "Understanding color models: A review," *ARPN Journal of Science and Technology*, vol. 2, no. 3, pp. 265–275, 2012.
- [2] R. Szeliski, *Computer Vision: Algorithms and Applications*, 2nd ed. Springer, 2022.
- [3] B. Ghanekar, L. R. Johnson, J. L. Laughlin, M. K. O'Malley, and A. Veeraraghavan, "Video-based surgical tool-tip and keypoint tracking using multi-frame context-driven deep learning models," in *Proc. IEEE 22nd Int. Symp. Biomedical Imaging (ISBI)*, 2025.
- [4] A. J. Martin, J. Atoum, and J. Y. Wu, "Challenges in generalizing vision-based surgical tool keypoint detection across surgical tasks and datasets," submitted to *SPIE Medical Imaging*, 2027.
- [5] C. Rother, V. Kolmogorov, and A. Blake, "'GrabCut': Interactive foreground extraction using iterated graph cuts," *ACM Transactions on Graphics*, vol. 23, no. 3, pp. 309–314, 2004.
- [6] B. Peng, L. Zhang, and D. Zhang, "A survey of graph theoretical approaches to image segmentation," *Pattern Recognition*, vol. 46, no. 3, pp. 1020–1038, 2013.
- [7] S. B. Kotsiantis, "Supervised machine learning: A review of classification techniques," *Informatica*, vol. 31, no. 3, pp. 249–268, 2007.
- [8] L. Bottou, F. E. Curtis, and J. Nocedal, "Optimization methods for large-scale machine learning," *SIAM Review*, vol. 60, no. 2, pp. 223–311, 2018.

- [9] University of Southampton, "Interactive optical flow demonstration," Java applet.
https://www.southampton.ac.uk/~msn/book/new_demo/opticalFlow/
- [10] "Network flow visualizer (max flow / min cut)," interactive web tool.
<https://web-apps.thecoatlessprofessor.com/graph-algorithms/network-flow.html>
- [11] OpenCV, "Interactive foreground extraction using GrabCut algorithm," OpenCV-Python Tutorials.